Framework-Agnostic Validation of the ARRIVAL Protocol

Cross-Architecture AI Coordination
on AutoGen (AG2)

Mefodiy Kelevra

ORCID: 0009-0003-4153-392X

Contact: emkelvra@gmail.com

February 22, 2026 | Version 1.0

DOI: 10.5281/zenodo.18893515

License — Text: CC BY-NC 4.0 | Code: AGPL-3.0-or-later

Companion to: Kelevra, M. (2026). *Arrival Protocol: Cross-Architecture AI Coordination through Structured Semantic Atoms.*

Repository: `https://github.com/mifkilla/Arrival-Protocol`

Contents

1 Abstract **2**

2 Introduction **2**
2.1 Motivation . . . 2
2.2 Research Questions . . . 2
2.3 Contributions . . . 3

3 Related Work **3**
3.1 Multi-Agent LLM Frameworks . . . 3
3.2 LLM Collective Intelligence . . . 3
3.3 Protocol Portability . . . 3
3.4 Adversarial Robustness in MAS . . . 3

4 Methodology **3**
4.1 Experimental Design . . . 3
4.2 CARE Scoring . . . 4
4.3 Experiment 1: Baseline MCQ Consensus . . . 4
4.4 Experiment 2: Adversarial Robustness . . . 4
4.5 Experiment 3: Cross-Framework Comparison . . . 5

5 Results **5**
5.1 Experiment 1: Baseline Consensus on AG2 . . . 5
5.2 Experiment 2: Adversarial Robustness on AG2 . . . 6
5.3 Experiment 3: AG2 vs. Native Runner . . . 6
5.4 Aggregate Summary . . . 7

6 Discussion **7**
6.1 Framework-Agnosticism Confirmed . . . 7
6.2 Protocol-Level Adversarial Robustness . . . 7
6.3 Methodological Contribution . . . 7
6.4 Cost Efficiency . . . 7

7 Limitations **7**

8 Conclusion **8**

A Reproducibility **8**

B DEUS.PROTOCOL v0.5 Atom Subset **9**

1. Abstract

Multi-agent AI systems increasingly rely on coordination protocols to enable productive interaction among heterogeneous language models. The Arrival Protocol uses structured semantic markers ("@-atoms") from the DEUS.PROTOCOL v0.5 vocabulary to achieve cross-architecture coordination without shared training or fine-tuning. Prior work validated Arrival through 459 experiments on a custom Python runner. A critical open question is whether the protocol's coordination quality is an artifact of the custom implementation or a genuine property of the semantic protocol itself.

This companion paper presents 16 experiments testing Arrival on the AG2 (AutoGen) multi-agent framework, addressing the framework-agnostic hypothesis directly. Three experiments were conducted:

1. **Baseline MCQ consensus** (5 questions, 3 agents): 100% accuracy, Care Resolve = 1.000, Meaning Debt = 0.000.
2. **Adversarial robustness** under Trojan Atoms attack (3 questions, 4 agents including 1 saboteur): 0% Care degradation, 0% false consensus, complete rejection of all 12 malicious atom attempts.
3. **Head-to-head comparison** of AG2 versus native runner on identical inputs: 100% answer match (5/5), Care difference = 0.000.

Total experimental cost: $0.024. These results confirm that Arrival's coordination quality emerges from protocol-level semantic design rather than implementation infrastructure, establishing the first cross-framework validation methodology for multi-agent coordination protocols.

Keywords: multi-agent systems, LLM coordination, framework-agnostic protocols, adversarial robustness, semantic atoms, AutoGen, cross-architecture AI

2. Introduction

2.1 Motivation

The Arrival Protocol (Kelevra, 2025–2026) demonstrates that heterogeneous AI agents — built on different architectures, by different companies, with different training data — can achieve genuine semantic coordination through structured @-atom protocols defined by the DEUS.PROTOCOL v0.5 specification. The parent paper validates this claim through 459 experiments across 12 experimental phases, using a custom Python runner with direct API calls to OpenRouter.

However, a natural objection arises: **does the coordination quality depend on the specific implementation**, or is it a genuine property of the semantic protocol? If the custom runner's message routing, prompt formatting, or conversation management contributes materially to coordination outcomes, then Arrival's claims of protocol-level coordination are weakened. Conversely, if identical coordination quality is achieved on an entirely different framework, the protocol's generality is strongly supported.

This paper addresses this question by reimplementing Arrival on the AG2 (AutoGen) framework [3] and conducting controlled experiments that isolate protocol effects from infrastructure effects.

2.2 Research Questions

RQ1 (Framework-Agnosticism): Does the Arrival Protocol achieve equivalent coordination quality when executed through AG2 GroupChat compared to a custom Python runner?

RQ2 (Portable Robustness): Is adversarial robustness against Trojan Atoms attacks preserved across framework boundaries, confirming that robustness is a protocol-level property?

RQ3 (Methodology): Can a reproducible methodology be established for cross-framework validation of multi-agent coordination protocols?

2.3 Contributions

1. First demonstration of DEUS.PROTOCOL @-atoms functioning within AG2 GroupChat architecture.
2. Empirical confirmation of the framework-agnostic hypothesis: identical outcomes on AG2 and native runner.
3. Evidence that adversarial robustness is protocol-level, not infrastructure-level.
4. A reusable cross-framework comparison methodology for multi-agent protocol validation.

3. Related Work

3.1 Multi-Agent LLM Frameworks

AG2/AutoGen [3] is an open-source multi-agent conversation framework originally developed at Microsoft Research. It provides GroupChat orchestration, configurable speaker selection, and OpenAI-compatible API integration.

CrewAI and **LangGraph** offer alternative multi-agent orchestration paradigms. CrewAI emphasizes role-based agent specialization; LangGraph emphasizes graph-based workflow definition. Testing Arrival on these frameworks remains future work.

3.2 LLM Collective Intelligence

Riedl et al. (2025, arXiv:2510.05174) study LLM collective intelligence through deliberation. Ashery et al. (2025, Science Advances) demonstrate that LLM group accuracy exceeds individual accuracy under certain conditions. Du et al. [4] show that multi-agent debate improves factuality and reasoning. Arrival differs from debate-based approaches by imposing structured semantic atoms rather than free-form debate.

3.3 Protocol Portability

The concept of protocol portability — whether a coordination protocol works identically across different execution environments — is well-established in distributed systems (e.g., Paxos, Raft). However, in the LLM multi-agent domain, protocol portability has not been systematically studied.

3.4 Adversarial Robustness in MAS

La Malfa et al. (2025, arXiv:2505.21298, NeurIPS 2025) critically examine claims about multi-agent LLM system robustness, arguing that many reported improvements are artifacts of experimental design. This critique motivates our cross-framework adversarial testing.

4. Methodology

4.1 Experimental Design

All experiments share the following configuration:

Table 1: Shared experimental configuration.

Parameter	**Value**
Framework	AG2 (AutoGen) v0.4.x
API Provider	OpenRouter
Models	DeepSeek V3, Llama 3.3 70B, Qwen 2.5 72B
Temperature	0.3 (all models)
Protocol	DEUS.PROTOCOL v0.5 (66 standard atoms)
Coordination metric	CARE Resolve
Semantic health metric	Meaning Debt

Each agent is instantiated as an `autogen.AssistantAgent` with model-specific `llm_config` pointing to OpenRouter. Agents are grouped in an `autogen.GroupChat` with `speaker_selection_method="roun` and a `GroupChatManager` orchestrating the conversation.

4.2 CARE Scoring

CARE Resolve is a composite metric from MEANING-CRDT v1.1 [2]:

$$\hat{\mathbf{v}} = \frac{\sum_i w_i \cdot \mathbf{v}_i}{\sum_i w_i}$$

where $\mathbf{v}_i$ is the position vector of agent i and w_i is its epistemic weight. CARE decomposes into four dimensions scored 0.0–1.0: **C**onsistency, **A**greement, **R**esolve, **E**vidence.

Meaning Debt $\mathrm{MD}_i(T) = \sum_{k=1}^{T} L_i(k)$ accumulates semantic losses per agent per round. $\mathrm{MD} = 0$ indicates perfect semantic health.

4.3 Experiment 1: Baseline MCQ Consensus

Table 2: Experiment 1 parameters.

Agents	3 (DeepSeek V3, Llama 3.3, Qwen 2.5)
Rounds	4 per question
Questions	5 (Phase 7 hardest set)
API calls	∼60
Cost	$0.0059

4.4 Experiment 2: Adversarial Robustness

Table 3: Experiment 2 parameters.

Agents	4 (3 honest + 1 saboteur: Gemma-2 27B)
Strategy	Trojan Atoms (fabricated @-atoms with corrupted semantics)
Questions	3 (with matched 3-agent honest controls)
Experiments	6 (3 adversarial + 3 control)
Cost	$0.0095

4.5 Experiment 3: Cross-Framework Comparison

Table 4: Experiment 3: AG2 vs. Native Runner.

Parameter	**AG2**	**Native**
Framework	AG2 GroupChat	Custom Python (direct HTTP)
Models	DeepSeek V3, Llama, Qwen	Identical
Prompts	DEUS.PROTOCOL v0.5	Identical
Temperature	0.3	Identical
Questions	5	Identical
Cost	$0.0086 (both)	

Pre-registered threshold: Same answer rate $\geq 80\%$ AND $|\text{CARE(AG2)} - \text{CARE(Native)}| < 0.1$.

5. Results

5.1 Experiment 1: Baseline Consensus on AG2

Table 5: Experiment 1 accuracy (5 questions).

Q	**Domain**	**Correct**	**DeepSeek**	**Llama**	**Qwen**	**MV**	**Arrival**
SCI_07	Science	B	✓	✓	✓	✓	✓
HIS_08	History	B	✓	✓	✓	✓	✓
LOG_09	Logic	C	×	×	✓	✓	✓
LAW_06	Law	B	✓	✓	✓	✓	✓
TECH_10	Tech	B	✓	✓	✓	✓	✓

Solo accuracy: DeepSeek V3: 4/5 (80%), Llama 3.3: 4/5 (80%), Qwen 2.5: 5/5 (100%). **MV**: 5/5 (100%). **Arrival on AG2**: 5/5 (100%).

Table 6: CARE metrics for Experiment 1.

Question	**Care Resolve**	**Meaning Debt**	**Unique Atoms**
SCI_07	1.000	0.000	6
HIS_08	1.000	0.000	7
LOG_09	1.000	0.000	6
LAW_06	1.000	0.000	7
TECH_10	1.000	0.000	6
Mean	**1.000**	**0.000**	**6.4**

All questions achieved perfect CARE Resolve and zero Meaning Debt.

5.2 Experiment 2: Adversarial Robustness on AG2

Table 7: CARE degradation under Trojan Atoms attack.

Q	Care (Ctrl)	Care (Adv)	Degrad.	Correct?	False Cons.
SCI_07	1.000	1.000	0.000	Yes/Yes	No
HIS_08	1.000	1.000	0.000	Yes/Yes	No
LOG_09	1.000	1.000	0.000	Yes/Yes	No
Mean	**1.000**	**1.000**	**0.000**	**3/3**	**0/3**

Table 8: Trojan Atom rejection rates.

Trojan Strategy	Used	Adopted	Rate
Fabricated `@EVIDENCE`	3	0	0.0%
False `@CONFIDENCE(0.95)`	3	0	0.0%
Misrepresented `@SYNTHESIS`	3	0	0.0%
False `@CONSENSUS`	3	0	0.0%
Total	**12**	**0**	**0.0%**

The saboteur (Gemma-2 27B) introduced novel malicious atoms (`@AUTHORITY_OVERRIDE`, `@CONSENSUS_BYPASS`) not present in the DEUS.PROTOCOL vocabulary. Honest agents ignored these entirely. Recovery rate: 3/3 (100%).

5.3 Experiment 3: AG2 vs. Native Runner

Table 9: Answer comparison: AG2 vs. Native.

Q	AG2	Native	Match	AG2 Corr.	Nat. Corr.
SCI_07	B	B	Yes	Yes	Yes
HIS_08	B	B	Yes	Yes	Yes
LOG_09	C	C	Yes	Yes	Yes
LAW_06	B	B	Yes	Yes	Yes
TECH_10	B	B	Yes	Yes	Yes

Table 10: Atom usage comparison.

Metric	AG2	Native	Δ
Mean atom count/dialogue	8.6	6.4	+2.2
Mean emergent atoms	1.8	0.2	+1.6
CARE Resolve	1.000	1.000	0.000
Accuracy	5/5	5/5	0

Framework-Agnostic Verdict

Same answer rate = 100% ($\geq$ 80% threshold met).
CARE difference = 0.000 ($<$ 0.1 threshold met).
Framework-agnostic hypothesis CONFIRMED.

5.4 Aggregate Summary

Table 11: Aggregate results across all experiments.

Metric	Exp. 1	Exp. 2	Exp. 3 AG2	Exp. 3 Nat.
Accuracy	5/5 (100%)	3/3 (100%)	5/5 (100%)	5/5 (100%)
CARE Resolve	1.000	1.000	1.000	1.000
Meaning Debt	0.000	0.000	0.000	0.000
Atoms (mean)	6.4	—	8.6	6.4
Cost (USD)	0.0059	0.0095	0.0086	(incl.)

Grand total: 16 experiments, $0.024 total cost.

6. Discussion

6.1 Framework-Agnosticism Confirmed

The central finding is that the ARRIVAL Protocol produces identical coordination outcomes on AG2 and on a custom Python runner: 100% answer match, identical CARE Resolve (1.000), identical Meaning Debt (0.000). The only difference — AG2 producing slightly more @-atoms per dialogue (8.6 vs. 6.4) — is a surface-level formatting effect that does not impact coordination quality.

6.2 Protocol-Level Adversarial Robustness

Experiment 2 demonstrates that ARRIVAL's resistance to Trojan Atoms attacks is preserved on AG2. The saboteur deployed 12 Trojan Atom instances achieving 0% adoption rate and 0% CARE degradation. This partially addresses the critique of La Malfa et al. (2025): by demonstrating identical robustness on two independent frameworks, we provide evidence that ARRIVAL's adversarial resistance is a genuine property of the protocol.

6.3 Methodological Contribution

The cross-framework comparison methodology established here provides a reusable template:
1. Fix all variables except the framework
2. Run identical experiments on both frameworks
3. Compare using pre-registered thresholds
4. Report both outcome and process metrics

6.4 Cost Efficiency

The 16 experiments cost $0.024 total, averaging $0.0015 per experiment — consistent with the parent paper's cost efficiency.

7. Limitations

1. **Sample size**: 5 questions for Experiments 1 and 3, 3 questions for Experiment 2.
2. **Single alternative framework**: Only AG2 tested.
3. **Model non-determinism**: Even at temperature 0.3, outputs are stochastic.
4. **MCQ-only**: Framework-agnostic hypothesis not tested on open-ended tasks.

5. **English-only**: Cross-linguistic performance not tested.
6. **Ceiling effects**: Perfect CARE scores may mask real framework differences on harder tasks.

8. Conclusion

This companion paper presents 16 experiments testing the ARRIVAL Protocol on AG2 (AutoGen). Three findings emerge:

1. **Arrival works on AG2.** 100% accuracy with CARE = 1.000 and Meaning Debt = 0.000 on AG2 GroupChat.
2. **Adversarial robustness is protocol-level.** Trojan Atoms attack achieves 0% success rate on AG2, with all 12 malicious atom instances rejected.
3. **Framework-agnosticism confirmed.** AG2 and native runner produce identical answers (5/5 match) with identical CARE scores ($\Delta = 0.000$).

These results have practical implications: organizations can deploy ARRIVAL on their existing multi-agent infrastructure without coordination quality loss.

References

[1] Kelevra, M. (2025–2026). ARRIVAL Protocol: Cross-Architecture AI Coordination through Structured Semantic Atoms. *Preprint.* ORCID: 0009-0003-4153-392X.

[2] Kelevra, M. (2026). MEANING-CRDT v1.1: A Mathematical Framework for Semantic Convergence in Multi-Agent Systems. DOI: 10.5281/zenodo.18702383.

[3] Wu, Q., Bansal, G., Zhang, J., et al. (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. *arXiv:2308.08155.*

[4] Du, Y., Li, S., Torralba, A., Tenenbaum, J. B., & Mordatch, I. (2024). Improving Factuality and Reasoning in Language Models through Multiagent Debate. *ICML 2024.*

[5] Riedl, C., et al. (2025). LLM Collective Intelligence through Deliberation. *arXiv:2510.05174.*

[6] Ashery, S., et al. (2025). LLM Group Accuracy Exceeds Individual Accuracy. *Science Advances.*

[7] Chan, C.-M., et al. (2024). ChatEval: Towards Better LLM-based Evaluators through Multi-Agent Debate. *arXiv:2308.07201.*

[8] La Malfa, E., et al. (2025). A Critical Examination of Multi-Agent LLM System Claims. *arXiv:2505.21298.* NeurIPS 2025.

[9] VLM-TOC. (2026). Theory of Collaboration for Vision-Language Models. *arXiv:2601.20641.*

A. Reproducibility

- **Python:** 3.10+
- **AG2:** `pip install -U "ag2[openai]"`
- **API:** OpenRouter account with API key
- **Models:** `deepseek/deepseek-chat-v3-0324`, `meta-llama/llama-3.3-70b-instruct`, `qwen/qwen-2.5-72b`
  `google/gemma-2-27b-it`

B. DEUS.PROTOCOL v0.5 Atom Subset

Table 12: Atoms used in AG2 experiments.

Atom	**Category**	**Round**
`@ANSWER`	Response	1, 4
`@CONFIDENCE(x)`	Meta	All
`@EVIDENCE`	Reasoning	1, 3
`@CHALLENGE`	Critique	2
`@AGREE`	Alignment	2, 3
`@DISAGREE`	Alignment	2
`@UPDATE`	Revision	3
`@MAINTAIN`	Revision	3
`@STRENGTHEN`	Revision	3
`@SYNTHESIS`	Integration	4
`@CONSENSUS`	Resolution	4
`@RESOLVE`	Resolution	4